Homework: text classification with Bag-of-Words and NBC and sentiment prediction with logistic regression

Goal of the assignment. The purpose of this homework is to understand the computational logic of two standard text-processing pipelines:

- a **text classifier based on Bag-of-Words and Multinomial Naive Bayes**,
- a **sentiment classifier based on manually extracted features and logistic regression**.

The submitted work must show clearly what happens during `fit` and what happens during `predict`. The point of the task is not to call a ready-made function, but to explain and compute the intermediate quantities that the model uses.

Allowed tools. Calculator, spreadsheet, or a small Python script used only to verify arithmetic. In particular, Python may be used to evaluate logarithms, exponentials, and the sigmoid function. However, all formulas, intermediate values, and conclusions must be shown in the submitted solution.

Not allowed. It is not allowed to use ready-made implementations such as `CountVectorizer`, `MultinomialNB`, `LogisticRegression`, or any library that performs the model fitting automatically.

Submission. Submit one file in **Teams**: either a **PDF** or a clear **scan of handwritten calculations**. The file must include intermediate calculations, short explanations, answers to the control questions, and final classification decisions.

Maximum score. The assignment is worth **100 points**.

1. Part A – Text classification with Bag-of-Words and Multinomial Naive Bayes

In this part the task is to classify very short reviews as `positive` or `negative`. Each document is represented as a Bag-of-Words count vector.

Training set

ID	Training text	Class
1	good fun	positive
2	great nice	positive
3	good great fun	positive
4	bad boring	negative
5	awful bad	negative
6	slow boring bad	negative

Preprocessing assumptions.

- All words are already lowercased.
- There is no stemming or lemmatization.
- Each token is separated by a blank space.
- The vocabulary is built **only from the training set**.
- Use **Laplace smoothing** with parameter $\alpha = 1$.

Suggested vocabulary. Build the vocabulary from the training set and keep the following token order:

$$V = (\texttt{good}, \texttt{fun}, \texttt{great}, \texttt{nice}, \texttt{bad}, \texttt{boring}, \texttt{awful}, \texttt{slow}).$$

Interpretation of the `fit` step. In Multinomial Naive Bayes, fitting means:

- counting how many training documents belong to each class,
- counting how many times each vocabulary token occurs inside each class,
- converting counts into class priors and conditional probabilities.

The class prior is computed as

$$P(c) = \frac{N_c}{N},$$

where N_c is the number of training documents in class c and N is the total number of training documents.

With Laplace smoothing, the conditional probability of token w in class c is

$$P(w \mid c) = \frac{\text{count}(w, c) + 1}{\sum_{v \in V} \text{count}(v, c) + |V|}.$$

Interpretation of the `predict` step. For a new document represented by token counts $n_w(x)$, the model computes a score proportional to the posterior probability:

$$\text{score}(c) = P(c) \prod_{w \in V} P(w \mid c)^{n_w(x)}.$$

Because products of many small probabilities become numerically inconvenient, you may also compute

$$\log \text{score}(c) = \log P(c) + \sum_{w \in V} n_w(x) \log P(w \mid c).$$

Both approaches lead to the same final class decision.

Tasks to complete

A1. Build the vocabulary and write the Bag-of-Words representation of every training document.

A2. Compute the class priors $P(\texttt{positive})$ and $P(\texttt{negative})$.

A3. For each class, count the total number of token occurrences and compute all smoothed conditional probabilities $P(w \mid c)$.

A4. Classify the following test documents:

$$x^{(1)} = \texttt{good great}, \qquad x^{(2)} = \texttt{aweful boring}, \qquad x^{(3)} = \texttt{good slow new}.$$

For each test document, show the class scores and the final predicted class.

A5. Explain in a short paragraph what exactly the model stores after `fit` and what operations it performs during `predict`.

Required interpretation

Your explanation must explicitly address the following points:

- why Bag-of-Words ignores word order,
- what Laplace smoothing changes and why it is needed,
- why the model can compare scores without computing the full posterior denominator,
- how the naive conditional independence assumption appears in the formula.

2. Part B – Sentiment prediction with feature extraction and logistic regression

In this part each text is transformed into a small numeric feature vector. Logistic regression is then used to predict whether the sentiment is positive ($y = 1$) or negative ($y = 0$).

Feature extraction rules

Use the following lexicons:

$$\texttt{POS} = \{\texttt{good}, \texttt{great}, \texttt{fun}, \texttt{nice}\}, \qquad \texttt{NEG} = \{\texttt{bad}, \texttt{awful}, \texttt{boring}, \texttt{slow}\}.$$

For every text extract exactly three features:

$$\begin{aligned} f_1 &= \text{number of words from the POS lexicon,} \\ f_2 &= \text{number of words from the NEG lexicon,} \\ f_3 &= \begin{cases} 1 & \text{if the text contains an exclamation mark !,} \\ 0 & \text{otherwise.} \end{cases} \end{aligned}$$

Training set

ID	Training text	Label y
1	good fun !	1
2	great nice	1
3	good great fun!	1
4	bad boring	0
5	awful bad	0
6	slow boring bad	0

Interpretation of the `fit` step. In logistic regression, fitting does not mean storing token frequencies. Instead, it means:

- extracting numeric features from each text,
- computing a score
$$z = w_1 f_1 + w_2 f_2 + w_3 f_3 + b,$$
- converting the score into a probability with the sigmoid function
$$\sigma(z) = \frac{1}{1 + e^{-z}},$$

- updating the model parameters w_1, w_2, w_3, b using the gradient of the loss.

To make the computations manageable by hand, use the following simplified training setting:

- initial weights: $w_1 = w_2 = w_3 = 0$,
- initial bias: $b = 0$,
- learning rate: $\eta = 1$,
- perform **exactly one batch gradient descent step** on the whole training set.

For a training set of size m, the gradients are

$$\frac{\partial J}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)x_{ij},$$
$$\frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i),$$

where

$$\hat{y}_i = \sigma(z_i), \qquad z_i = w^T x_i + b.$$

The update rule is

$$w_j \leftarrow w_j - \eta\frac{\partial J}{\partial w_j}, \qquad b \leftarrow b - \eta\frac{\partial J}{\partial b}.$$

Interpretation of the `predict` step. For a new text, first extract features (f_1, f_2, f_3), then compute z, then compute $\sigma(z)$. If $\sigma(z) \geq 0.5$, predict class 1 (positive); otherwise predict class 0 (negative).

Tasks to complete

B1. Extract the three features for every training text and write the feature matrix.

B2. Using the initial parameters $w = (0, 0, 0)$ and $b = 0$, compute z_i and $\hat{y}_i$ for all training examples.

B3. Compute the gradients $\partial J/\partial w_1$, $\partial J/\partial w_2$, $\partial J/\partial w_3$, and $\partial J/\partial b$.

B4. Perform one batch gradient descent update and write the new parameter values.

B5. Using the updated parameters, classify the following test texts:

$$t^{(1)} = \texttt{good nice !}, \qquad t^{(2)} = \texttt{bad slow}, \qquad t^{(3)} = \texttt{good bad}.$$

For each test text, show the extracted features, the value of z, the probability $\sigma(z)$, and the final predicted label.

B6. Explain in a short paragraph what the model stores after `fit` and how this differs from the Bag-of-Words Naive Bayes model from Part A.

Required interpretation

Your explanation must address the following points:

- why logistic regression outputs a probability through the sigmoid function,
- what the sign of a weight means,
- how the bias changes the decision threshold,
- why feature extraction is necessary before logistic regression can process text.

3. Control questions

Answer all questions in complete sentences or short paragraphs. Single keywords will not receive full credit.

C1. What exactly does the Bag-of-Words representation preserve, and what information does it lose?

C2. What quantities are stored by Multinomial Naive Bayes after the `fit` step?

C3. Why is Laplace smoothing important in text classification?

C4. Why can log-scores be used instead of raw probability products in Naive Bayes?

C5. In Part A, what is the meaning of the vocabulary, and why must it be fixed after training?

C6. What quantities are stored by logistic regression after the `fit` step?

C7. Why must raw text be transformed into numeric features before logistic regression can be applied?

C8. What is the role of the sigmoid function in logistic regression?

C9. What does a positive weight mean, and what does a negative weight mean?

C10. Why is one gradient descent step enough for this homework, but not enough in a real training process?

4. Grading criteria – 100 points

Assessment component	**Points**
Part A: correct vocabulary and Bag-of-Words representations	5
Part A: correct class priors and smoothed conditional probabilities	10
Part A: correct classification of the three test documents with calculations	20
Part A: conceptual explanation of `fit` and `predict`	5
Part B: correct feature extraction for all training texts	5
Part B: correct computation of logits and probabilities before the update	5
Part B: correct gradients and updated parameters	15
Part B: correct classification of the three test texts with calculations	10
Part B: conceptual explanation of `fit` and `predict`	5
Control questions	20

Important. Missing intermediate steps, formulas, or explanations will reduce the score even if the final class label is correct.

5. Expected structure of the submitted solution

The submitted work should have the following structure:

1. header or title page with name, surname, and group,
2. Part A: vocabulary, Bag-of-Words tables, probability calculations, and predictions,
3. Part B: extracted features, one gradient update, and predictions,
4. answers to the control questions,
5. a brief final comparison of the two models.

The final comparison should make clear that

- Naive Bayes learns by counting class frequencies and token frequencies,
- logistic regression learns numeric weights for extracted features,
- in both models the `predict` step transforms the input text into a representation and then computes a class-dependent score.